自动驾驶技术第三次实验报告

nuScenes mini

LiDAR BEV 三维目标检测与 PointPillars

姓名： 杜承敖

学号： 2024302112019

专业： 计算机科学与技术（雷军班）

课程： 自动驾驶技术

武汉大学

2026 年 10 月 13 日

目录

摘要 **2**

1 实验目的与数据 **3**

2 LiDAR 点云与图像：区别、优势及互补 **3**

3 PointPillars 原理与代码理解 **4**

3.1 从点云输入到柱特征 5

3.2 BEV 映射、二维网络与检测头 6

4 训练设置、断点恢复与 Loss 曲线 **7**

4.1 训练设置与损失曲线 7

4.2 训练过程 Log：实际运行摘录 8

5 验证结果与阈值解释 **9**

6 LiDAR BEV 结果展示 **11**

6.1 微调后的 LiDAR BEV：固定验证帧 0 11

6.2 微调后的 LiDAR BEV：固定验证帧 1 12

6.3 微调后的 LiDAR BEV：固定验证帧 2 13

摘要

本报告记录基于 PointPillars 的 LiDAR BEV 三维目标检测实践。实验使用 nuScenes mini 的 404 个关键帧，按官方场景划分为 323 帧训练数据和 81 帧验证数据。首先比较点云与图像的表达方式、优势和互补关系；随后结合 HardVFE、BEV 映射、二维骨干网络及锚框检测头源码理解处理流程。在 RTX 4060 Laptop GPU 上，从公开权重开始微调检测头，完成 1 轮训练后恢复至累计 5 轮，共 1615 个批次。整轮平均损失由 2.873274 降至 2.456259。使用同一验证集，NDS 从 0.026860 提升至 0.118891，mAP 从 0.000061 提升至 0.062302。报告包含真实训练 Log、Loss 曲线、参数更新审计和三帧 BEV 预测图，并说明教学范围与 mini 数据的局限。

关键词：nuScenes mini；LiDAR；PointPillars；柱特征；三维目标检测

1 实验目的与数据

按课程指导书，先评估公开 PointPillars 权重，再进行检测头微调：训练 1 轮后从检查点恢复至累计 5 轮，最后在同一 mini 验证集比较指标，并显示 LiDAR 点云及预测框。

复用实验二的 nuScenes mini 原始 JSON 元数据，并从实验一的本地点云压缩包提取对应的全部 404 个关键帧。重新生成 MMDetection3D 1.4 格式的 LiDAR 索引，未使用 PETR 模型、相机索引或预测结果，也未下载额外数据集。

表 1: 运行环境与数据设置

项目	实际设置
GPU	NVIDIA GeForce RTX 4060 Laptop GPU
Python	3.9.25
PyTorch / CUDA	2.0.1+cu118 / 11.8
MMCV / MMEngine	2.1.0 / 0.10.7
MMDetection / MMDetection3D	3.2.0 / 1.4.0
数据划分	8 个训练场景 / 323 帧；2 个验证场景 / 81 帧；无交叉
输入	单 LiDAR 关键帧；原始五维存储，模型读取 x/y/z/intensity
感知区域 / BEV	x、y 为 [-25,25] 米；z 为 [-5,3] 米；200 × 200 网格

404 个点云文件均存在且数据有限；标注采用 LiDAR 坐标、长度/宽度/高度和 yaw，并保存 LiDAR 坐标速度。所有三维框中心回转至全局坐标后的最大误差为 4.547e-13 米。

未读取历史 sweeps 或相机图像。数据按官方 mini_train/mini_val 场景划分；验证集不完整覆盖全部十个检测类别。

2 LiDAR 点云与图像：区别、优势及互补

点云和图像描述同一交通场景，但信息组织方式不同。图像在像素网格上记录外观，LiDAR 根据激光回波测得空间采样点。点云中的 intensity 是回波强度，不能直接当作相机的 RGB 颜色。

表 2: LiDAR 点云与图像数据的比较

比较维度	LiDAR 点云	相机图像
数据表达	稀疏、无固定点序的三维点集合；本次输入为 x/y/z/intensity。	规则的二维像素网格，通常含 RGB 三通道，细节密集。
几何与距离	直接提供距离和三维位置，便于确定物体尺寸、朝向和障碍物空间。	像素主要表示投影；单幅普通 RGB 图像不能直接给出唯一的绝对深度。
外观与语义	能反映三维轮廓、反射强度，但颜色、文字和细纹理信息有限。	颜色、纹理、交通灯状态、标志文字和目标外观信息丰富。
采样与遮挡	远处及小目标往往点数较少，遮挡后可见表面采样更少。	远处目标仍可能有可辨外观，但像素变少、遮挡也会影响识别。
光照与天气	主动测距，对环境可见光变化相对不敏感；雨雾、反射特性和遮挡仍可影响回波。	成本相对低、图像分辨率高；弱光、眩光、雨雾和曝光变化会降低可用信息。

各自优势：LiDAR 更适合提供有物理尺度的空间几何，图像更适合提供密集外观和细粒度语义。例如，点云有助于判断前车距离与占据空间，相机有助于辨认车辆外观和交通灯颜色。任何一种传感器都不能仅凭自身保证覆盖所有困难场景。

互补方法：先统一时间戳和坐标系，用外参把点云变换到相机坐标，再用内参投影到图像。在相机前方且可见的点处采样图像特征或语义类别概率，与点的几何特征拼接，随后进行三维检测；也可分别检测后在统一坐标中关联目标。

```
p_camera = T_camera_from_lidar @ [x, y, z, 1]
u = fx * X_camera / Z_camera + cx
v = fy * Y_camera / Z_camera + cy
# Require Z_camera > 0 and a valid visible image position.
```

代码清单 1: 投影关系的简化表达：还需处理畸变、遮挡和不同采集时刻的车辆运动。

PointPainting [2] 是这种互补思路的例子：将图像分割类别分数附加到对应点云，再送入点云检测器。本次课程模型仅使用 LiDAR，未实现相机融合；此处说明的是可扩展的融合原理。

[2] Vora 等，PointPainting，CVPR 2020。原文链接

3 PointPillars 原理与代码理解

PointPillars 的关键思路是把无规则点云组织成竖直柱，学习每柱的特征，再映射为规则 BEV 伪图像，使后续网络可以使用高效的二维卷积 [1]。本实验采用课程提供的 HardVFE 编码实现；以下解释以实际配置和源码为准。

以下源码路径相对于 third_party/mmdetection3d-1.4.0/mmdet3d/models/；课程配置与冻结策略分别见 course_common.py 和 course_modules.py。

3.1 从点云输入到柱特征

course_common.py 的 single_frame_pipeline() 使用 LoadPointsFromFile(load_dim=5, use_dim=4)：原始文件按五维点记录解析，送入模型的是前四维。训练阶段同步对点云和真实框旋转、缩放、翻转，并过滤范围和类别；验证阶段没有随机增强，也没有加载历史 sweeps。

柱划分：voxel_size=[0.25,0.25,8]，感知范围为 50×50×8 米，因此 x/y 各 200 格，z 只有一格。虽然框架称此操作为 voxelization，本配置得到的是贯穿高度范围的柱。每柱最多 64 点；训练最多保留 6000 个柱，验证最多 8000 个，空缺点通过补零组织成张量。

表 3: 柱特征的组织形式

符号 / 设置	含义
K	该帧实际保留的非空柱数量，随点云变化。
[K,64,4]	柱中点的原始输入组织形式；另有 num_points 保存真实点数。
10 维点特征	4 维输入 + 到柱内点均值的 3 维偏移 + 到柱中心的 3 维偏移。
[K,64]	最终每个柱的 64 维学习特征。

```
points_mean = (
    features[:, :, :3].sum(dim=1, keepdim=True) /
    num_points.type_as(features).view(-1, 1, 1))
# TODO: maybe also do cluster for reflectivity
f_cluster = features[:, :, :3] - points_mean
features_ls.append(f_cluster)
```

代码清单 2: 源码摘录：HardVFE.forward()，voxel_encoders/voxel_encoder.py 第 409-414 行。

上述均值只除以真实点数，f_cluster 描述柱内局部形状；f_center 描述点相对于固定网格中心的位置。拼接后的补零位置由 get_paddings_indicator() 生成的 mask 清零，防止把补齐位置当作观测点。高度 z 仍保留在特征中，使用 BEV 并不表示完全丢掉高度。

HardVFE 中两层 VFELayer 使用共享线性变换、BatchNorm 和 ReLU。第一层把点特征与该柱的最大池化特征拼接，第二层再池化得到一个 64 维柱向量；这将不同点数的柱变为等长表示。本配置的特征装饰是 10 维，不能套用其他实现的固定维数。

固定验证帧 0 的实际前向记录：范围过滤后输入 32636 个四维点，形成 3550 个非空柱。第一层输入为 [3550, 64, 10]，输出为 [3550, 64, 128]；第二层输出为 [3550, 64]。

[1] Lang 等，PointPillars，CVPR 2019。原文链接

3.2 BEV 映射、二维网络与检测头

```
if not self.with_pts_bbox:
    return None
voxel_features = self.pts_voxel_encoder(voxel_dict['voxels'],
                                        voxel_dict['num_points'],
                                        voxel_dict['coors'], img_feats,
                                        batch_input_metas)
batch_size = voxel_dict['coors'][-1, 0] + 1
x = self.pts_middle_encoder(voxel_features, voxel_dict['coors'],
                            batch_size)
x = self.pts_backbone(x)
if self.with_pts_neck:
    x = self.pts_neck(x)
return x
```

代码清单 3: 源码摘录：MVXTwoStageDetector.extract_pts_feat()，detectors/mvx_two_stage.py 第 207-219 行。

这段调用链依次完成柱编码、BEV 映射、骨干特征提取和多尺度融合。PointPillarsScatter 为每帧创建全零画布，柱坐标按 [batch,z,y,x] 保存；使用 y*nx+x 计算平面位置，将对应柱特征写入画布。空柱维持零值，恢复为 [B,64,200,200]，这里的通道是学习特征，并非 RGB。

```
batch_mask = coors[:, 0] == batch_itt
this_coors = coors[batch_mask, :]
indices = this_coors[:, 2] * self.nx + this_coors[:, 3]
indices = indices.type(torch.long)
voxels = voxel_features[batch_mask, :]
voxels = voxels.t()

# Now scatter the blob back to the canvas.
canvas[:, indices] = voxels
```

代码清单 4: 源码摘录：PointPillarsScatter.forward_batch()，middle_encoders/pillar_scatter.py 第 85-93 行。

SECOND 骨干用二维卷积逐级降低空间分辨率、扩大感受野；SECONDFPN 对三个尺度分别处理并对齐到同一网格，再拼接为 384 通道。Anchor3DHead 用三个 1×1 卷积分支输出类别、三维框编码和方向分类，解码后通过旋转 NMS 去除重复框。

表 4: 固定验证帧 0 的中间特征尺寸

实际单帧前向记录	输出尺寸
HardVFE	[3550, 64]
PointPillarsScatter	[1, 64, 200, 200]
SECOND 三尺度	[[1, 64, 100, 100], [1, 128, 50, 50], [1, 256, 25, 25]]
SECONDFPN	[[1, 384, 100, 100]]
分类分支	[1, 140, 100, 100]
框回归分支	[1, 126, 100, 100]
方向分支	[1, 28, 100, 100]

上表是在第 5 轮权重上对固定验证帧 0 额外执行一次 eval 前向得到的，未更新模型。每个网格位置有 14 个锚框；分类维度为锚框数 ×10，回归维度为锚框数 ×9（含位置、尺寸、yaw、vx/vy 编码），方向维度为锚框数 ×2。完整记录见 model_shape_trace.json。

代码中的 loss_cls 为 FocalLoss，loss_bbox 为 SmoothL1Loss，loss_dir 为方向交叉熵（权重 0.2）；训练 Log 的 loss 是已加权损失之和。course_modules.py 按参数名仅打开 pts_bbox_head.* 的梯度，所以三个卷积分支的 weight/bias 共 6 个张量参加参数更新。冻结参数不代表 BatchNorm 的运行统计缓冲区也固定。

4 训练设置、断点恢复与 Loss 曲线

4.1 训练设置与损失曲线

从已校验 SHA256 的公开 f19d00a3 权重初始化。仅检测头参数参与优化；Pillar 编码器、骨干及 neck 参数冻结。单 GPU、batch=1、AMP、AdamW，学习率 0.0001，权重衰减 0.01，梯度裁剪 max_norm=35，随机种子 20261012。

表 5: 首轮训练和断点续训审计

运行	完成轮次 / 批次数	训练审计耗时	参数更新 / 梯度张量
首次训练	第 1 轮 / 323	20.50 秒	6/6；6
断点续训	第 2-5 轮 / 1292	56.39 秒	6/6；6

两段审计均为 PARAMETERS_UPDATED，存在真实梯度和参数更新。两段运行的检查点分别确认完成第 1 轮和累计第 5 轮；检查全部模型浮点权重及 1615 个训练损失点均为有限值。续训恢复优化器及训练进度，–epochs 5 表示累计五轮。

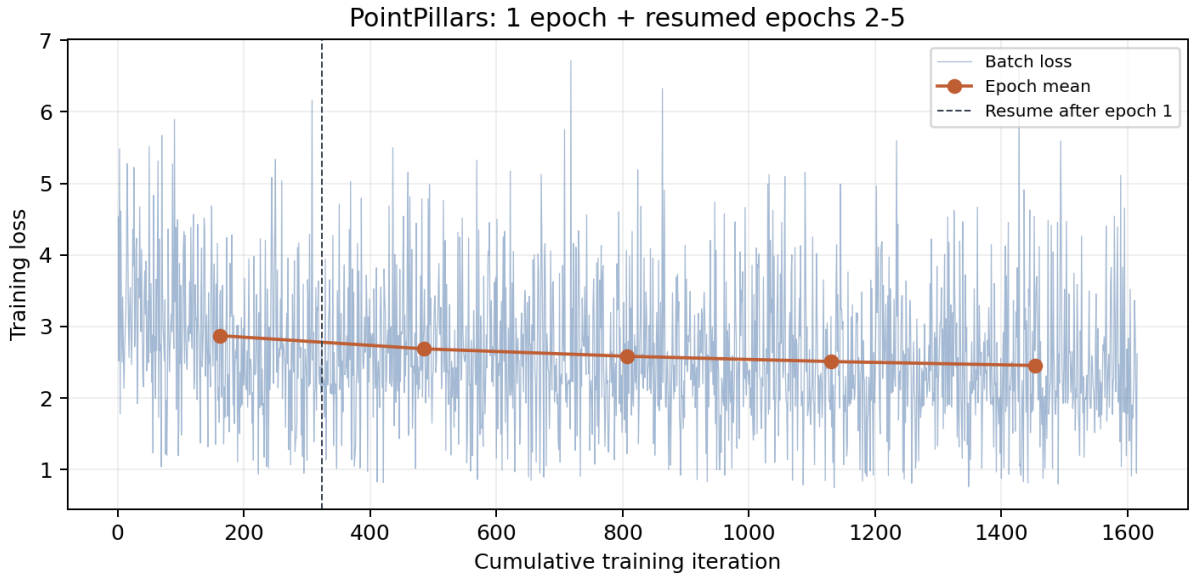


图 1: 五轮累计损失。虚线为第 1 轮后断点恢复位置；橙色点为每轮全部 323 个批次损失均值。

表 6: 各轮全部批次的平均训练损失

轮次	1	2	3	4	5
平均损失	2.87327	2.68935	2.58483	2.51116	2.45626

损失下降反映训练优化，不等同于验证精度提升。第 1 轮及续训损失曲线、训练日志、梯度审计和检查点均单独保留。训练审计耗时包含该训练运行内部的验证等开销；逐步命令耗时另见 execution.json。

AMP 审计记录 3 次非有限缩放梯度事件；该计数属于动态损失缩放诊断，不能表述为所有原始梯度均有限。本次最终参数和训练损失检查通过。

4.2 训练过程 Log：实际运行摘录

以下内容直接来自本次保存的控制台日志，页面仅自动折行，未重新构造数值。完整日志位于 outputs/setup/run1_console.log 和 run5_console.log，训练目录内另有框架日志及结构化记录。

```
10/13 20:00:58 - mmengine - INFO - Epoch(train) [1][ 10/323]  lr: 1.0000e-04  eta: 0:00:38
    time: 0.1220  data_time: 0.0133  memory: 295  grad_norm: 2.7219  loss: 3.2840  loss_cls:
    1.1094  loss_bbox: 1.9927  loss_dir: 0.1818
10/13 20:01:11 - mmengine - INFO - Epoch(train) [1][320/323]  lr: 1.0000e-04  eta: 0:00:00
    time: 0.0419  data_time: 0.0095  memory: 313  grad_norm: 2.4906  loss: 2.5351  loss_cls:
    0.9419  loss_bbox: 1.4118  loss_dir: 0.1814
```

代码清单 5: 首轮训练：第 10 和第 320 个批次。

```
10/13 20:01:33 - mmengine - INFO - resumed epoch: 1, iter: 323
10/13 20:01:33 - mmengine - INFO - Epoch(train) [2][ 10/323]  lr: 1.0000e-04  eta: 0:01:32
    time: 0.0476  data_time: 0.0082  memory: 279  grad_norm: 2.4806  loss: 2.5871  loss_cls:
    0.9739  loss_bbox: 1.4223  loss_dir: 0.1910
10/13 20:02:21 - mmengine - INFO - Epoch(train) [5][320/323]  lr: 1.0000e-04  eta: 0:00:00
    time: 0.0352  data_time: 0.0038  memory: 294  grad_norm: 2.7527  loss: 2.5562  loss_cls:
    0.8707  loss_bbox: 1.5123  loss_dir: 0.1732
```

代码清单 6: 断点恢复：恢复到 epoch=1、iter=323，继续第 2 轮并完成第 5 轮。

```
10/13 20:01:17 - mmengine - INFO - AUDIT: PARAMETERS_UPDATED; changed=6; max_change=2.187e-02
10/13 20:02:29 - mmengine - INFO - AUDIT: PARAMETERS_UPDATED; changed=6; max_change=8.097e-02
```

代码清单 7: 两段训练的参数更新审计原文。

Log 读法：Epoch(train) [N][i/323] 表示第 N 轮、第 i 个批次；lr 是学习率，time/data_time 是运行记录中的迭代/数据时间，memory 是日志记录的显存值；loss_cls、loss_bbox、loss_dir 分别对应分类、框回归和方向损失。日志中的损失汇总由框架日志处理器输出，单条批次记录不能替代整轮均值。

断点恢复不是重新初始化模型：train.py 将 resume=True 与 load_from 指向 epoch_1.pth，Runner 恢复权重、优化器和累计迭代状态。日志明确出现 resumed epoch: 1, iter: 323，而后出现第 2 轮记录；最终检查点元数据为 epoch=5，证明累计训练五轮。

全部批次记录的第 1 轮平均 loss 为 2.87327，第 5 轮为 2.45626，下降约 14.51%。各批次受点数、目标分布及数据增强影响而波动，因此结合完整曲线和轮均值观察训练。3 次缩放梯度非有限事件已保留在审计中，最终权重及损失有限性检查通过。

```
python train.py --epochs 1 --work-dir outputs/run1_compact --amp
python train.py --resume-from outputs/run1_compact/epoch_1.pth --epochs 5 --work-dir
    outputs/run5_compact --amp
python test.py --checkpoint outputs/run5_compact/epoch_5.pth --output-dir
    outputs/finetuned_compact --vis-count 3 --score-thr 0.10
```

代码清单 8: 实际训练、断点续训与最终验证命令。

5 验证结果与阈值解释

公开权重和第 5 轮权重均对同一清单的全部 81 个 mini_val 帧完成推理与 nuScenes 检测评估。根据预测中的样本索引和点云路径回查，81 个 token 及顺序均一致；mini 无独立带标注测试集，本报告的测试为验证集评估。

表 7: 微调前后 NDS 与 mAP

指标	公开权重	累计 5 轮	变化（微调减基线）
NDS	0.026860	0.118891	+0.092031
mAP	0.000061	0.062302	+0.062240

表 8: 微调前后检测误差

误差指标	公开权重	累计 5 轮
mATE	1.15690	0.85550
mASE	0.90030	0.68340
mAOE	1.26350	0.95130
mAVE	1.68880	1.38360
mAAE	0.83150	0.63240

mAP 汇总不同中心距离阈值下的类别 AP；NDS 综合精度和位置、尺寸、朝向、速度、属性误差。mATE 单位米，mAOE 单位弧度，mAVE 单位米/秒。部分类别的误差项不适用或因匹配不足使用默认值，不宜解释为精确测得的单个目标误差。

表 9: 两个展示阈值下的预测框数量

固定验证帧	展示阈值 0.10	展示阈值 0.30
帧 0	51	2
帧 1	45	4
帧 2	77	4

阈值由 0.10 提高至 0.30，仅对缓存预测进行展示筛选，不重新运行模型，也不改变已计算的 NDS/mAP。BEV 图仅含原始点云和模型预测框，没有叠加真实框；框的数量或视觉密集程度不能替代检测评估。

本次微调的 NDS 变化为 +0.092031，mAP 变化为 +0.062240。结果仅对应 mini、单帧及 50×50 米教学范围，不能与完整 nuScenes 榜单直接比较；公开权重也不能视作从未见过相关数据的独立测试模型。

6 LiDAR BEV 结果展示

6.1 微调后的 LiDAR BEV：固定验证帧 0

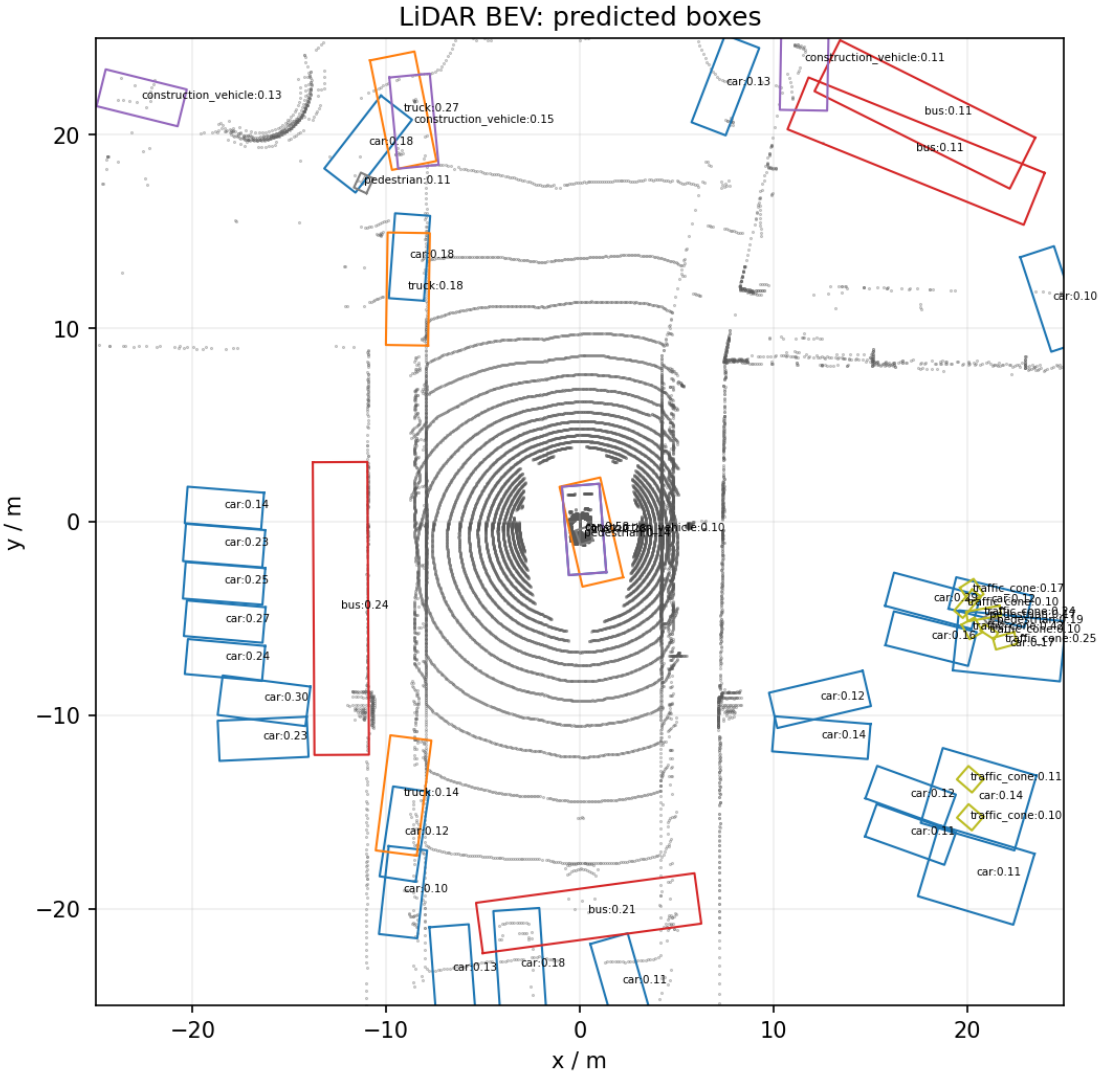


图 2: 累计五轮模型，展示阈值 0.10，共 51 个预测框。

灰点为点云在 LiDAR x-y 平面的投影；彩色矩形表示预测三维框的底面和朝向，文字为类别及置信度。坐标单位为米，显示范围为 ±25 米。该视图未叠加真实框，需结合前述验证指标判断效果。

相同点云在阈值 0.30 下显示 2 个框。两套图片均保留，文件夹分别为 visualizations_thr010 和 visualizations_thr030。

点云来源：n008-2018-08-01-15-16-36-0400___LIDAR_TOP___1533151603547590.pcd.bin

6.2 微调后的 LiDAR BEV：固定验证帧 1

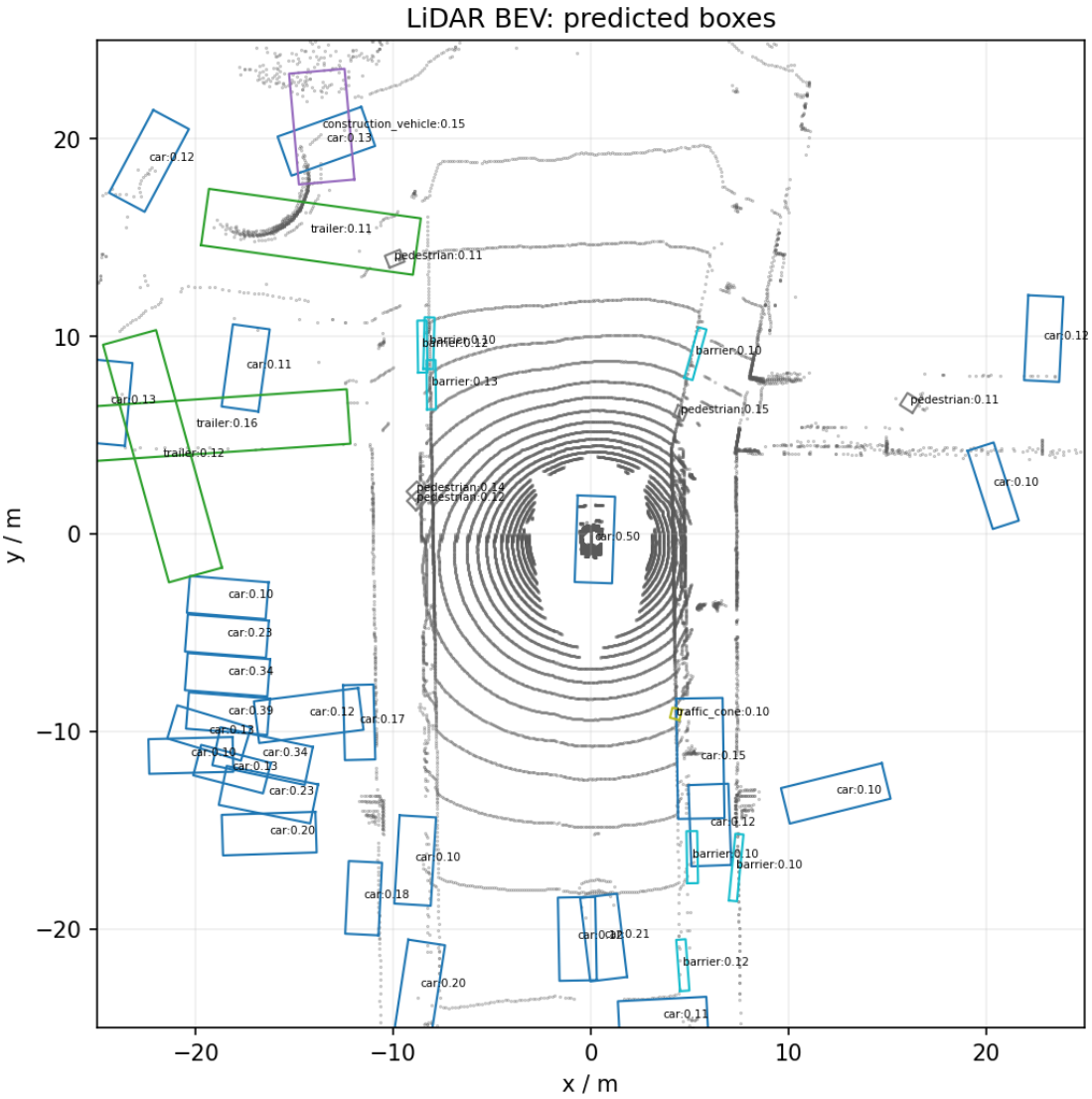


图 3: 累计五轮模型，展示阈值 0.10，共 45 个预测框。

灰点为点云在 LiDAR x-y 平面的投影；彩色矩形表示预测三维框的底面和朝向，文字为类别及置信度。坐标单位为米，显示范围为 ±25 米。该视图未叠加真实框，需结合前述验证指标判断效果。

相同点云在阈值 0.30 下显示 4 个框。两套图片均保留，文件夹分别为 visualizations_thr010 和 visualizations_thr030。

点云来源：n008-2018-08-01-15-16-36-0400__LIDAR_TOP__1533151604048025.pcd.bin

6.3 微调后的 LiDAR BEV：固定验证帧 2

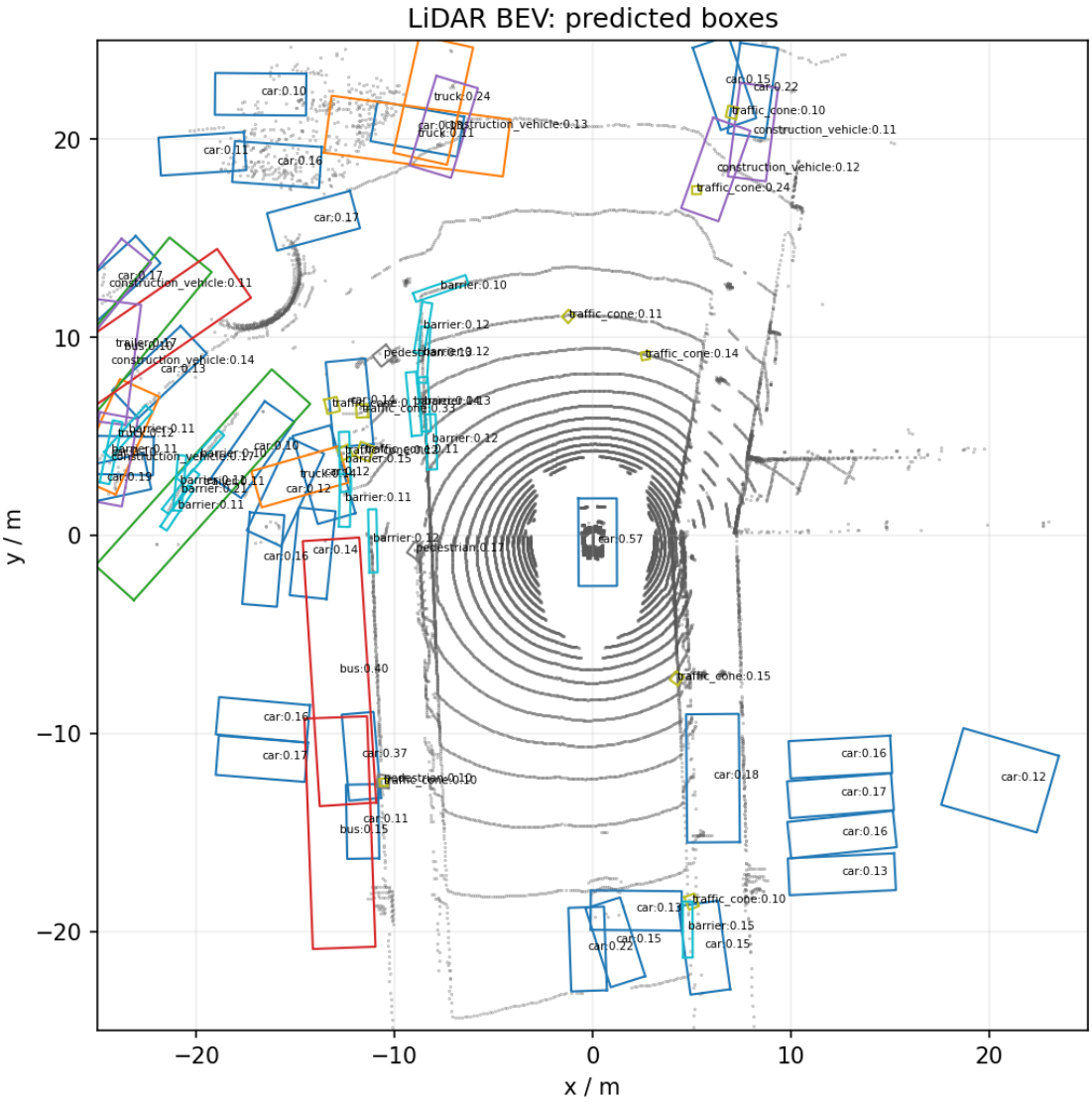


图 4: 累计五轮模型，展示阈值 0.10，共 77 个预测框。

灰点为点云在 LiDAR x-y 平面的投影；彩色矩形表示预测三维框的底面和朝向，文字为类别及置信度。坐标单位为米，显示范围为 ±25 米。该视图未叠加真实框，需结合前述验证指标判断效果。

相同点云在阈值 0.30 下显示 4 个框。两套图片均保留，文件夹分别为 visualizations_thr010 和 visualizations_thr030。

点云来源：n008-2018-08-01-15-16-36-0400___LIDAR_TOP___1533151604547893.pcd.bin

复现及证据：提交包中的 README.md 给出完整命令；manifest.json、comparison.json、final_validation.json、训练审计、损失图、完整日志及 epoch_5.pth 均随包提交。参考：课程指导书、随附 MMDetection3D 1.4.0 源码与 nuScenes SDK。